N-Queens

https://www.geeksforgeeks.org/problems/n-queen-problem0315/1

Place N queens on an N×N chessboard so that no two queens attack each other. A queen attacks along its row, column and diagonals.

Place one queen in each row. This prevents row conflicts.

For the current row, try every column.

Check earlier queens: same column, same diagonal.

If safe, place the queen and recurse to the next row.

If later placement fails, remove this queen and try another column.

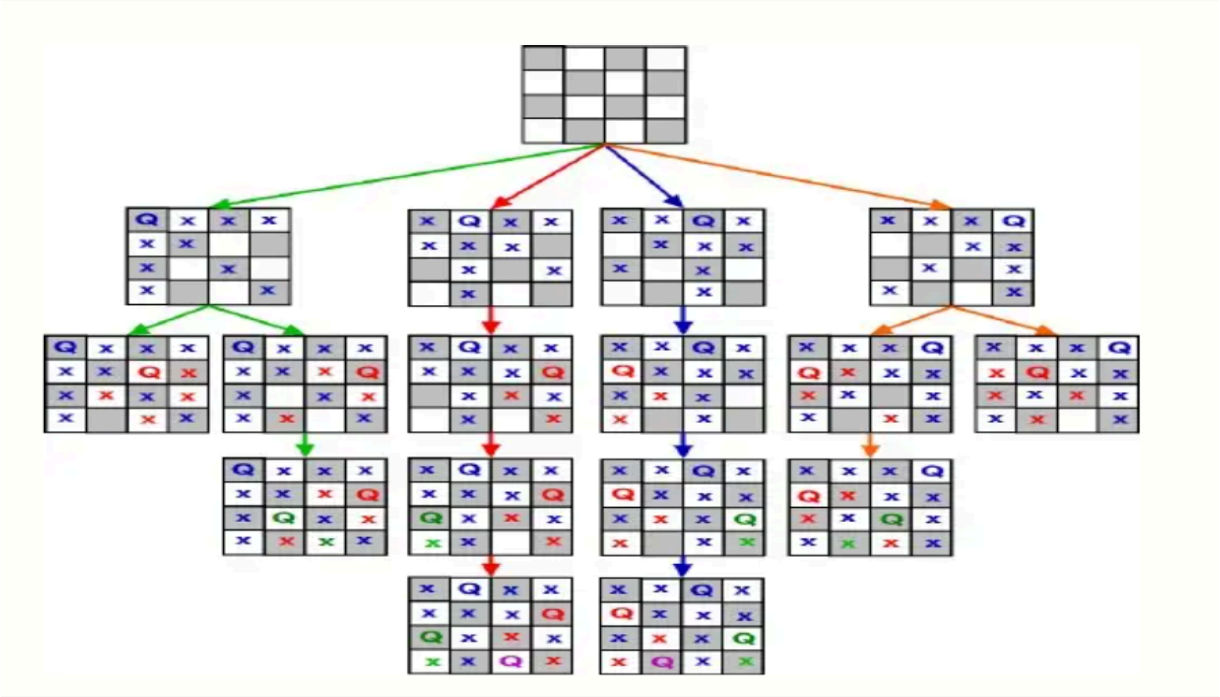
If row == N, a complete valid arrangement has been found.

```
 int n;
vector<int> queenCol; // column chosen for each row

bool safe(int row, int col) {
    for (int r = 0; r < row; r++) {
        int c = queenCol[r];
        if (c == col) return false;
        if (r - c == row - col) return false;
        if (r + c == row + col) return false;
    }
    return true;
}

void solve(int row) {
    if (row == n) {
        // A complete solution is found.
        for (int r = 0; r < n; r++) cout << queenCol[r] << " ";
        cout << endl;
        return;
    }
    for (int col = 0; col < n; col++) {
        if (safe(row, col)) {
            queenCol[row] = col; // choose
            solve(row + 1);      // explore
            queenCol[row] = -1;  // undo
        }
    }
}
```

This version prints all solutions as column positions, one per row. For N=4 there are 2 solutions. The simple safety check scans earlier rows; it is designed for learning.



Rat in a Maze

https://www.geeksforgeeks.org/problems/rat-in-a-maze-problem/1

A rat is placed at the top-left cell of an N × N maze and must reach the bottom-right cell.

1 means the cell is open; the rat may enter it.

0 means the cell is blocked; the rat cannot enter it.

Start = (0,0); destination = (N−1,N−1).

This implementation allows four moves: Down, Left, Right and Up.

A cell cannot be visited more than once in the same path.

We will find ALL valid paths and represent each path as a string of D, L, R and U.

```
bool solve(int r, int c) {
    if (r < 0 || c < 0 || r >= n || c >= n) return false;
    if (maze[r][c] == 0 || visited[r][c]) return false;
    if (r == n - 1 && c == n - 1) return true;

    visited[r][c] = true;
```

```
    path.push_back('R');
    if (solve(r, c + 1)) return true;
    path.pop_back();

    path.push_back('D');
    if (solve(r + 1, c)) return true;
    path.pop_back();

    visited[r][c] = false; // undo this cell choice
    return false;
}
```

Mark a cell when it becomes part of the current path.

Do not enter a blocked cell or a cell already visited in this path.

When the recursive exploration returns, unmark the cell so another path can use it.

visited is not a permanent record that a cell can never be used again. It only prevents revisiting within the current path.

Topic	Core idea	Remember
Merge Sort	Divide, sort, merge	O(n log n) in every case
Quick Sort	Pivot and partition	Average O(n log n), worst $O(n^2)$
Backtracking	Choose, explore, undo	Prune invalid paths
N-Queens	Place queens row by row	Check columns and diagonals
Rat in a Maze	Explore valid routes	Bounds, blocked, visited